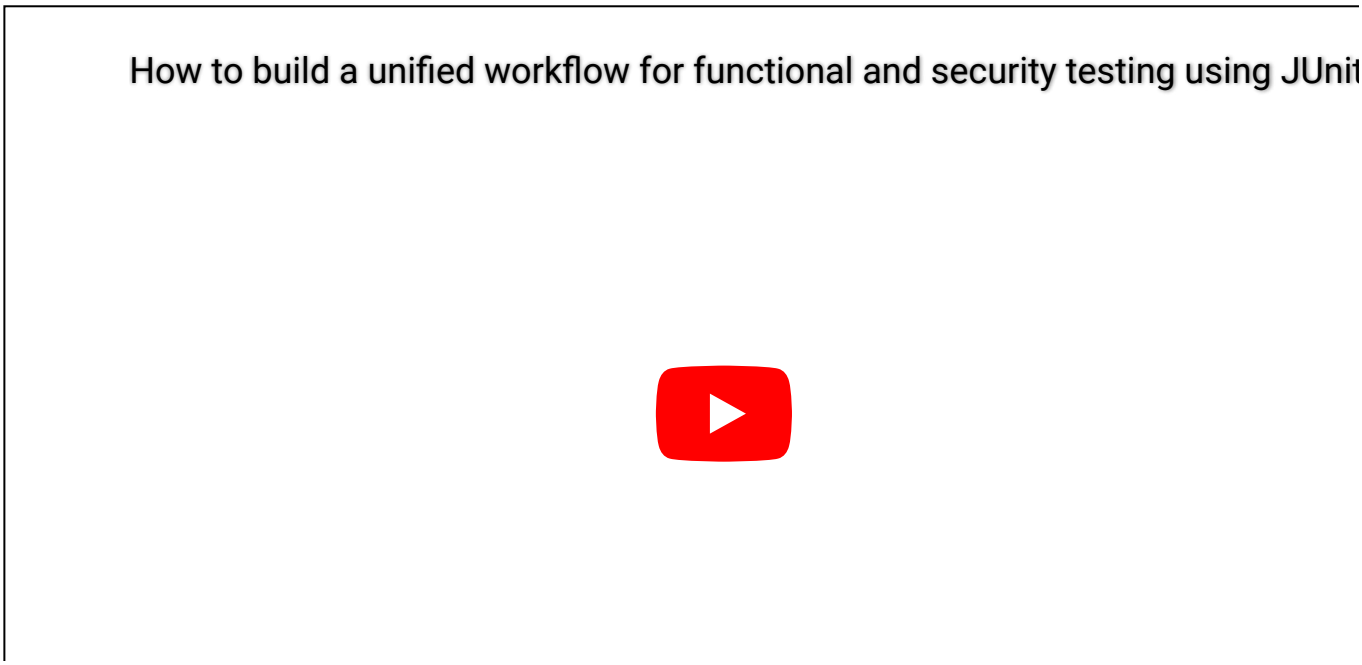
code intelligence

Josh Grant | 8 min read

Unit Testing Vs Fuzz Testing - Two Sides of the Same Coin?

Most developers, including myself, have written unit tests before. Fuzz testing on the other hand has only started seeing widespread industry usage in recent years. Yet, some voices are already praising fuzz testing as the more effective approach, due to its ability to automatically generate negative and invalid test inputs. Let's put this claim to the test and see how these two approaches match up.



Unit testing vs Fuzz testing

code intelligence

tests and for verifying if a piece of code functions correctly given a specific input.

Fuzz testing on the other hand is an automated, non-deterministic testing approach that requires domain knowledge and experience to be deployed efficiently. But when done right, it can be very powerful at uncovering bugs and vulnerabilities that are deeply hidden within the source code. Due to its complexity and specialized nature, fuzz testing is often done by dedicated security teams. Instead of asserting how a system should behave, fuzzers explore how it should definitely not behave, by automatically generating a ton of invalid inputs and seeing if any of them will trigger undesired behavior.

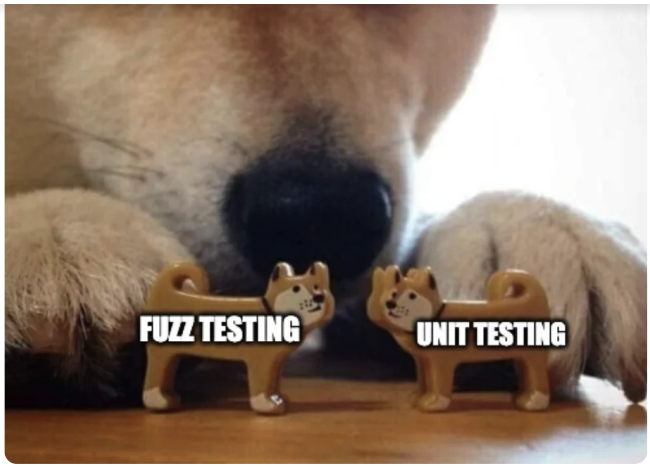
Fuzz testing is a form of negative testing as it investigates how a program behaves given invalid or unexpected inputs. Meanwhile, unit testing is a form of positive testing, as it investigates a program's behavior given valid inputs. Unit tests are usually done by devs, while fuzzing is traditionally done by security teams.

Why Unit Testing and Fuzzing Belong Together

Functional and security issues can both be highly consequential in Java applications (e.g. downtime, data breaches, UX issues, etc). For this reason, unit testing and fuzz testing should not be substituted for one another. They should be built into one unified workflow, where they can run alongside each other: unit tests to find functional bugs and fuzz tests to detect security issues.

This would not only make it easier for developers to find and fix issues earlier, but it would also reduce friction between teams, as the line between functional and security issues is often blurry anyways. Imagine you find a bug that causes your software to malfunction, and the resulting crash exposes your application to security issues. Would this be considered a functional or security bug? With a unified workflow, developers would be empowered to fix all issues fast, regardless if they are functional bugs

code intelligence



How to Do It: One Unified Workflow

Fuzz testing and unit testing are two sides of the same coin. In an ideal world, developers would have the right tooling to do both, without any expert knowledge. Thanks to easy-to-use tools such as JUnit testing is already there. Fuzz testing is a different story. To make use of fuzz testing, many security teams still use DIY open-source tools that require plenty of hands-on configuration. To make it as easy as unit testing, my team and I have built an integration that allows developers to add fuzz tests to their JUnit tests effortlessly. Instead of @test, this allows you to call a fuzz test using @fuzztest.

How It Works

We integrated our CLI tool called CI Fuzz CLI into the JUnit testing framework. It runs on macOS, Linux and Windows and comes with support for common IDEs and build systems. If you want to test Java code with CI Fuzz CLI, the only requirements are Java JDK (f.e., OpenDesk or Zulu) and any build system

To download CI Fuzz CLI, run the installation script in GitHub.

```
sh -c "$(curl -fsSL
```

Example: Finding Security Bugs in a Java Library

code intelligence

ExploreMe.java

```java
package com.example;
public class ExploreMe {
// Function with multiple paths that can be discovered by a fuzzer.
public static void exploreMe(int a, int b, String c) {
if (a >= 20000) {
if (b >= 2000000) {
if (b - a < 100000) {
// Create reflective call
if (c.startsWith("@")) {
String className = c.substring(1);
try {
Class.forName(className);
} catch (ClassNotFoundException ignored) {
}
}
}
}
}
}
}
```

Let's use CI Fuzz CLI to write a fuzz test that can trigger this RCE.

Setting Up a Fuzz Test in 3 Easy Steps

In this video, I explained how you can get CI Fuzz CLI up and running in Maven. Further below, I also added the corresponding code examples.

code intelligence

Instructions and download links are also available in our documentation.

How to set up CI Fuzz CLI in Maven

The commands in CI Fuzz CLI will interactively guide you through the needed options and provide instructions on what to do next. To see the full list of cifuzz commands, including all options and parameters, use cifuzz command --help.

1. Init

The first step is to initialize the project you want to test with CI Fuzz CLI.

This step will vary depending on the build system you are working with. To modify relevant build files for your Gradle or Maven project, you will need to edit the build.gradle file in Gradle projects, or the pom.xml file in Maven. The process for creating a fuzz test is the same regardless of the build system being used.

To create a Maven project, you can use the cifuzz init command in the root directory of your project.

code intelligence

```
<version>0.13.0</version>
<scope>test</scope>
</dependency>

<dependency>
<groupId>org.junit.jupiter</groupId>
<artifactId>junit-jupiter-engine</artifactId>
<version>5.9.0</version>
<scope>test</scope>
</dependency>
```

2. Create

To create a fuzz test using CI Fuzz, you can run the cifuzz create java command in the current directory. This will create a stub for a fuzz test, which you can use as a starting point for your own test.

Although you can put the fuzz test anywhere, it's best practice to stay close to the code under test as you would with a regular unit test. In the sample Maven project in the CI Fuzz CLI repository, we created the test in src/test/java/com/example/FuzzTestCase.java.

```
FuzzTestCase.java
package com.example;
Import com.code_intelligence.jazzer.api.FuzzedDataProvider;
import com.code_intelligence.jazzer.junit.FuzzTest;
public class FuzzTestCase {
@FuzzTest
void myFuzzTest(FuzzedDataProvider data) {
int a = data.consumeInt();
int b = data.consumeInt();
String c = data.consumeRemainingAsString();
ExploreMe.exploreMe(a, b, c);
```

about this fuzz test:

- The test is located in the same package as our target class

code intelligence

input into multiple parts and organize them into different data types

- To run this fuzz test with cifuzz, you need to use the following command: "cifuzz run com.example.FuzzTestCase" from the project directory. This is the name that cifuzz recognizes for the test.

3. Run

Start the fuzz test by executing cifuzz run FuzzTestCase. CI Fuzz CLI now builds the fuzz test and starts a fuzzing run.

```
$ cifuzz run FuzzTestCase
[...]

Use ‘cifuzz finding <finding name>’ for details on a finding.

💥[awesome_gnu] Security Issue: Remote Code Execution in exploreMe (com.example.ExploreMe:13)

Note: The crashing input has been copied to the seed corpus at:
src/test/resources/com/examples/MyClassFuzzTestInputs/awesome_gnu

It will now be used as a seed input for all runs of the fuzz test, including remote runs with
artifacts created via ‘cifuzz bundle’ and regression tests. For more information, see:
https://github.com/CodeIntelligenceTesting/cifuzz#regression-testing

Execution time: 3s
Average exec/s: 316880
Findings: 1
New seeds: 5 (total: 5)
```

Coverage Reporting and Debugging

```
[awesome_gnu] Security Issue: Remote Code Execution in exploreMe (com.example.ExploreMe:13)
Date: 2022-11-10 13:31:07.94532426 +0100 CET
```

code intelligence

```
remote code execution depending on available classes on the classpath.
at jaz.Zer.<clinit>(Zer.java:54)
at java.base/java.lang.Class.forName0(Native Method)
at java.base/java.lang.Class.forName(Class.java:375)
at com.example.ExploreMe.exploreMe(ExploreMe.java:13)
at com.example.FuzzTestCase.myFuzzTest(FuzzTestCase.java:13)
== libFuzzer crashing input ==
MS: 0 ; base unit: 0000000000000000000000000000000000000000

0x40,0x6a,0x61,0x7a,0x2e,0x5a,0x65,0x72,0x0,0x1,0x0,0x40,0x0,0x0,0x0,0x75,
@jaz.Zer\000\001\000@\000\000\000u
artifact_prefix='./'; Test unit written to
./crash-f29a1020fa966baaf8c2326a19a03b73f8e5a3c9
Base64: QGphei5aZXIAAQBAAAAAdQ==
```

We also get a report on the number of lines and functions found and how many branches were covered.

```
Coverage Report
File | Functions Hit/Found | Lines Hit/Found | Branches Hit/Found
FuzzTestCase.java | 2/2 (100.0%) | 9/9 (100.0%) | 0/0 (100.0%)
src/explore_me.java | 1/1 (100.0%) | 23/23 (100.0%) | 8/8 (100.0%)
| | |
| Functions Hit/Found | Lines Hit/Found | Branches Hit/Found
total | 3/3 | 32 | 8/8
```

Optimizing your fuzzer for code coverage is highly advisable. It will enable the fuzzer to explore your application by taking many different paths through your code. This way, the fuzzer can learn if specific inputs or data structures are required to execute the code. The more information the fuzzer gathers, the more effective your tests will be.

Final Thoughts

code intelligence

Try CI Fuzz CLI

Related Articles

Dae Glendowne | 1 min read

Another Expression DoS Vulnerability Found in Spring - CVE-2023-20863

We found another Expression DoS vulnerability in Spring (CVE-2023-20863). CVSS Base Score: 7.5 (high). Here's everything you ...

Start Reading →

code intelligence

Daniel Teuchert | 3 min read

Integrating Fuzzing Into Automotive Security

Fuzzing has taken the automotive software industry by storm, Let's have a look at how integrating fuzzing helps automotive ...

Start Reading →

Khaled Yakdan | 2 min read

Level Up Your Unit Tests: How to Turn a JUnit Test into a Fuzz Test

code intelligence

Product

Product Tour

Start Fuzzing

CVEs & Findings

Monthly Demo

Careers

About Us

Open Positions

Contact

Press

Resources

What Is Fuzz Testing?

Documentation

Blog

Get in Touch

Contact Us

info@code-intelligence.com

Code Intelligence

Rheinwerkallee 6

53227 Bonn, Germany

Sign Up Our Newsletter

Follow Us

Imprint Privacy Policy